

Algorithm and Methods of Programming

TEAM WORK

MAZE GRAPH ALGORITHMS - FINAL PROJECT

Student Name:

Ngoc Bao Tram Tran 231ADB294

Ilaha Habibova 231ADB231

Kamalesh Ashokkkumar Kowsalya 221ADB216

1. Division of Work:

- Tram: Implemented Subtask A (BFS), Subtask D (Max Flow), maze parsing and validation
- Kamalesh: Implemented Subtask B (Dijkstra), Subtask E (MST), benchmarking extension
- Ilaha: Implemented Subtask C (comparison), JSON/GraphML export, testing and documentation

All members contributed to debugging, testing, and report writing.

2. Maze Representation:

The maze has been stored in a text file (maze_10x10_A.txt). Each line in the text file represents one row of the maze, and each character represents one cell. Cells with X represent that the path is blocked. The program reads the file line by line and stores the result in a list of strings named grid [r][c], where r and c represent the character in row and column from the text file.

The program validates the text file before running any algorithm to check whether the file is not empty and all rows have the same length; checks that the grid is at most 100x100; checks there is only one S (start) and G (goal); and produces a clear, error-free message without any failure.

Original Maze - maze_10x10_A.txt:

```
S1234X6789
1X2X4X6X89
11111X1111
X3XXX4XXX1
1234543211
1XXX5XXX21
```

```

1111511111
9X9X5X9X91
1234512341
9876X4321G

```

How we interpret it:

	0	1	2	3	4	5	6	7	8	9
0	S	1	2	3	4	X	6	7	8	9
1	1	X	2	X	4	X	6	X	8	9
2	1	1	1	1	1	X	1	1	1	1
3	X	3	X	X	X	4	X	X	X	1
4	1	2	3	4	5	4	3	2	1	1
5	1	X	X	X	5	X	X	X	2	1
6	1	1	1	1	5	1	1	1	1	1
7	9	X	9	X	5	X	9	X	9	1
8	1	2	3	4	5	1	2	3	4	1
9	9	8	7	6	X	4	3	2	1	G

Each character represents one cell: ‘S’ = start (value 0); ‘G’ = goal (value 0); ‘X’ = wall (not passable); ‘0’-‘9’ = passable cell with numeric value

Basic Information: Size: 10 rows × 10 columns, Number of wall cells (X): 24 cells; Number of movable cells: 80 cells (including S and G)

There are 24 blocked cells (X): (0,5), (1,1), (1,3), (1,5), (1,7), (2,5), (3,0), (3,2), (3,3), (3,4), (3,6), (3,7), (3,8), (5,1), (5,2), (5,3), (5,5), (5,6), (5,7), (7,1), (7,3), (7,5), (7,7), (9,4). There are 9 high value cells - dangerous if using cost model: (0,9): 9, (1,9): 9, (7,0): 9, (7,2): 9, (7,6): 9, (7,8): 9, (9,0): 9 - (1,8): 8, (9,1): 8 - (9,2): 7 - (9,3): 6

3. Graph Representation:

Each cell is uniquely identified by its coordinates (r, c), where r is the row index, c is the column index, both starting from 0. For example, in the provided maze_10x10_A.txt:

- START cell: ‘S’ - Coordinates: (r=0, c=0) - Meaning: row 0, column 0 (top left corner)
- GOAL cell: ‘G’ - Coordinates: (r=9, c=9) - Meaning: row 9, column 9 (bottom right corner)
- [Wall cell example] First wall cell: ‘X’ - Coordinates: (r=0, c=5) - Meaning: row 0, column 5
- [Passable cell example] Cell with value 4: ‘4’ - Coordinates: (r=0, c=4) - Meaning: row 0, column 4

Each non-wall cell (r, c) is a vertex. Two vertices are connected by an edge if movement between their corresponding cells is allowed according to the selected movement mode:

- For 4-directional: edges connect cells up/down/left/right
- For 8-directional: edges also connect diagonal neighbors (up-left, up-right, down-left, down-right)

4. Completed Subtasks:

Each of the five subtasks were completed and explained below along with the selected methods.

4.1 Subtasks A – Shortest path by Number of moves

4.1.1. Chosen Approach

We implemented Breadth-First Search (BFS) to solve this problem. The reason that we chose BFS as it explores nodes level by level, ensuring that the first time we reach the goal, we have found the shortest path in terms of number of moves. Since every move has equal cost (1), BFS is optimal and simpler than weighted-path algorithms like Dijkstra. BFS guarantees finding the shortest path in unweighted graphs.

4.1.2. Time Complexity

Time Complexity: $O(V + E)$ with V = Number of passable cells (vertices); E = Number of edges (neighbor connections). We visit each vertex only once (because of the visited set) and each edge is examined only once during neighbor exploration, so the total operations done are $O(V + E)$. In particular for maze_10x10_A.txt, $V = 76$ and $E = 194$ (4-directional) or 342 (8-directional). In general for an $n \times n$ maze, both V and E are $O(n^2)$, since the number of cells is proportional to the area, and each cell has at most 8 (4 and 8) neighbors, so $O(V + E) = O(n^2)$.

4.1.3. Space Complexity

The algorithm maintains three data structures: a visited set, a parent dictionary and a BFS queue, each of which can contain at most V elements, where V is the number of passable cells. For maze_10x10_A.txt, this is 76 cells. For a general $n \times n$ maze, since $V = O(n^2)$, the space complexity is $O(n^2)$.

4.2 Subtasks B – Minimum-Cost Path

4.2.1. Chosen Approach

For Subtask B we used Dijkstra's algorithm with a binary heap priority queue. The algorithm begins at the starting cell S with cost 0 and then initializes all other cells with initial cost as infinity, which means that it has not been visited yet. At all times the cell with the lowest known cost is processed first, using a priority queue. The algorithm analyzes each cell and determines the new costs to the neighbors for each cell using the selected cost model. If this new cost is lower than the previous known cost for that neighbor, then the cost is updated and the neighbor is inserted into the heap. This is repeated until the goal cell G is removed from the heap and the algorithm, terminated by transversing the path back to S following parent pointers.

There are three cost models that were implemented and tested. Model 1 is the entering cost (the cost of moving from cell u to cell v is the value of cell v). Model 2 is the leaving cost (the cost of moving from cell u to cell v is the values of cell u). Model 3 is the sum of cell values (combined cost). We choose Cost Model 1 as the best natural one, as it is a cost that you must pay when u enter a cell, such as a toll fee. The algorithm was executed both with 4-directional and 8-directional movement, with minimum cost of 26 and 18 respectively, for Cost Model 1.

4.2.2. Time Complexity

Dijkstra's algorithm's time complexity, using a binary heap, is $O((V+E) \log V)$;

Where V – number of cells that can be passed thorough,

E – number of edges between neighboring cells.

Each cell is inserted and deleted at most once and each heap operation requires $O(\log V)$ time since the heap needs to be reordered after every insertion and removal. Total number of edges is at most E, each relaxation may cost $O(\log V)$ for the heap, those $O(E)$ such cost will sum up to $O(E \log V)$. The cost processing all the vertices adds $O(V \log V)$ to make it $O((V + E) \log V)$.

This is quite efficient for our 10x10 maze with 76 passable cells and 194 directed edges in 4-dimensional mode, which is not particularly large and means our algorithm only does a few thousand operations at most.

4.2.3. Space Complexity

This maze has space complexity of $O(V)$. The algorithm works with 3 data structures: a distance dictionary to keep the best known cost to reach a cell, a parent dictionary to keep track of which cell a cell was reached from (for path reconstruction), and the priority queue heap. There is one entry for each of these structures in each passable cell, meaning that each structure will be $O(V)$ in space. Overall, it is an $O(V + E)$ solution, with the heap growing to $O(E)$ worst case and potentially allowing duplicate entries. But in a grid maze, the number of edges E is proportional

to V since each cell has at most 4 or 8 neighbors, which reduces the $O(V + E)$ algorithm to $O(V)$ in practice.

4.3 Subtasks C – Movement Mode Comparison

4.3.1. Chosen Approach

To compare the results, we reused BFS from Subtask A for the shortest path by number of moves and Dijkstra's algorithm from Subtask B for the minimum-cost path. For the cost comparison, we used Cost Model 1: Entering Cost, where the cost of moving from cell u to cell v is the value of cell v . This model was selected because it naturally represents paying the cost when entering a new maze cell.

For maze_10x10_A.txt, diagonal movement changed both results. The shortest path decreased from 18 moves in 4-directional mode to 11 moves in 8-directional mode, and the minimum cost decreased from 26 to 18. The results also show that the shortest path is not always the cheapest path: in 4-directional mode, the shortest path has 18 moves but cost 51, while the cheapest path has 20 moves but cost only 26. This happens because BFS minimizes moves, while Dijkstra minimizes total cost.

4.3.2. Time Complexity

Subtask C runs BFS and Dijkstra's algorithm for both 4-directional and 8-directional movement.

BFS has time complexity: $O(V + E)$

Dijkstra's algorithm has time complexity: $O((V + E) \log V)$

Since Dijkstra's algorithm is more expensive, the overall time complexity of Subtask C is: $O((V + E) \log V)$

where V is the number of passable cells and E is the number of edges between neighboring passable cells. For maze_10x10_A.txt, $V = 76$, $E = 194$ for 4-directional movement, and $E = 342$ for 8-directional movement.

For a general $n \times n$ maze, both V and E are $O(n^2)$, so the complexity can be written as: $O(n^2 \log n)$

4.3.3. Space Complexity

BFS uses a visited set, a parent dictionary, and a queue. Each can store at most V cells, so BFS requires: $O(V)$

Dijkstra's algorithm uses a distance dictionary, a parent dictionary, and a priority queue. In general, this requires: $O(V + E)$

Therefore, the overall space complexity of Subtask C is: $O(V + E)$

For a grid maze, E is proportional to V , so this can be simplified to: $O(V)$

For a general $n \times n$ maze, the space complexity is: $O(n^2)$

4.4 Subtasks D – Maximum Flow from G to S

4.4.1. Chosen Approach

We implemented the Edmonds-Karp algorithm which is a BFS-based implementation of the Ford-Fulkerson method for computing maximum flow. The Edmonds-Karp algorithm was chosen because it guarantees termination and polynomial time complexity by always choosing the shortest augmenting path (in terms of number of edges) using BFS. Unlike generic Ford-Fulkerson which can run indefinitely with irrational capacities, Edmonds-Karp always terminates in $O(V \cdot E^2)$ time with integer capacities. The algorithm is also simpler to implement correctly compared to push-relabel or other advanced flow algorithms, while still being efficient enough for our maze sizes (up to 100×100). It constructs a residual graph where the capacities are assigned based on the values of the cells and repeatedly finds augmenting paths from source (G) to sink (S) using BFS. For each augmenting path found, it pushes flow along the path equal to the minimum residual capacity along the path and updates the residual capacities. If no augmenting paths can be found anymore, we have found our maximum flow.

4.4.2. Time Complexity

Time Complexity: $O(V \cdot E^2)$ where V = number of passable cells (vertices) and E = number of directed edges. Each BFS to find an augmenting path takes $O(E)$ time since we may traverse all edges in the worst case. The number of iterations is at most $O(V \cdot E)$ because each iteration adds at least 1 unit of flow, and the total flow cannot exceed the sum of all edge capacities. Therefore, the total time is $O(V \cdot E)$ iterations multiplied by $O(E)$ per iteration, giving us $O(V \cdot E^2)$. In particular for maze_10x10_A.txt, we have $V = 76$ passable cells and $E = 194$ (4-directional) or $E = 342$ (8-directional), giving a theoretical worst case of $O(76 \cdot 194^2) \approx 2.8$ million operations for 4-directional movement, $O(76 \cdot 342^2) \approx 8.9$ million operations for 8-directional movement. For a general $n \times n$ maze, both V and E are proportional to n^2 , thus $O(V \cdot E^2) = O(n^6)$ in the worst case. However, in practice the algorithm terminates much faster because the number of augmentations is typically much smaller than the theoretical bound.

4.4.3. Space Complexity

Space Complexity: $O(V + E)$, where V = number of passable cells and E = number of edges. The algorithm uses few data structures: adjacency lists to represent the flow network, residual capacity matrices stored as nested dictionaries for sparse representation, and the original capacities to calculate final flow values, all of which take $O(V + E)$ space. The BFS structures

(visited set, parent dictionary, queue) take $O(V)$ space and are reused between iterations. For maze_10x10_A.txt, this implies $V = 76$ and $E = 194-342$. For a general $n \times n$ maze, as both V and E are proportional to n^2 , the space complexity is $O(n^2)$.

4.5 Subtasks E – Minimum spanning tree

4.5.1. Chosen Approach

In Subtask E, we calculated Minimum Spanning Tree on the graph of the maze using Prim's algorithm and binary heap priority queue. The Maze is modeled as an undirected weighted graph, with each of the non-wall cells being a vertex, and each of the connection between two adjacent passable cells being an edge. A cell edge will have a weight equal to the sum of the values of the two cells u and v that it connects. The algorithm begins at cell S and expands the spanning tree, one cell at a time, by always choosing the least costly edge that brings a cell into the tree that is not already in the tree.

The algorithm is as follows. The tree starts with a cost of zero and Cell S is added to the tree first. All edges from S to its neighbor are added to the heap sorted by weight cheapest first. The heap is then processed, by popping the cheapest edge. The algorithm first determines if the destination cell is already in set `in_tree` before inserting it. The edge is omitted if it has already been added to the tree, thus preventing cycles. If the target tree is not in the tree, it will be added to the tree, the edge will be stored in the edge list, the weight will add to the total and all edges leading to an unvisited adjacent cell will be added to the edge list. This is repeated until the heap is empty. The minimum spanning tree of the connected component containing S is the result. This algorithm also determines if the goal cell G has been reached, giving a check for connectivity. Our required maze was generated to be MST of 76 passable cells (50 edges) and the total weight was 390 in 4-directional mode and 370 in 8-directional mode. S could be reached by Goal G by both modes of movement.

4.5.2. Time Complexity

Prim's algorithm using a binary heap has a time complexity of $O((V + E) \log V)$;

Where V – number of cells that can be passed,

E – number of undirected edges between neighboring cells

Each cell added to `in_tree` once and is processed at most once. On insertion into the tree, all its edges are visited, and may be inserted into the heap, resulting in $O(\log V)$ time per heap insertion. The total number of edge examination is $O(E)$, each of them costs $O(\log V)$ which is the heap operation, so total cost is $O(E \log V)$. The final bound is $(V + E) \log V$ by adding the $O(V \log V)$ cost of the final popping of all the vertices from the heap. In practice, this is very fast for our 10x10

maze, where there are 76 cells that can be passed through and 194 cells that have neighboring directions to pass through 4-directional mode.

4.5.3. Space Complexity

Space Complexity is $O(V + E)$. The algorithm uses a variety of data structures during its operations. One set in_tree contains at most v entries, one corresponding to each cell that has been placed in the spanning tree. The best_key dictionary stores to each V entry the cheapest known (minimum possible) edge weight to reach the corresponding cell. The list of edges in MST is represented by mst_edges which have at most $V-1$ elements, as a spanning tree with V vertices always has $V-1$ edges. The worst-case heap size is the largest, up to E entries; this can happen if each edge examination adds an entry to the heap but not removes any other entries. This results in $O(E)$ space for heap. The total space is $O(V + E)$ and, since in a grid maze each cell has at most 4 or 8 neighbors, it simplifies to $O(V)$ in practice for grid-structured maze like ours.

Discussion of Results:

The results show that different graph algorithms give different “best” solutions because each subtask has a different objective.

For Subtask A, BFS found the shortest path by number of moves. The shortest path required 18 moves in 4-directional mode and 11 moves in 8-directional mode. This shows that diagonal movement can reduce the path length.

For Subtask B, Dijkstra’s algorithm found the minimum-cost path using Cost Model 1, where the cost is paid when entering a cell. The minimum cost was 26 in 4-directional mode and 18 in 8-directional mode. This means diagonal movement can also help avoid high-cost cells.

For Subtask C, the results show that the shortest path is not always the cheapest path. In 4-directional mode, the shortest path had 18 moves but cost 51, while the cheapest path had 20 moves but cost only 26. BFS minimizes moves, while Dijkstra minimizes total cost.

For Subtask D, the maximum flow from G to S was 1 in 4-directional mode and 4 in 8-directional mode. The 8-directional graph has more edges, so it provides more possible routes for flow.

For Subtask E, Prim’s algorithm found the MST of the connected component containing S. The MST weight was 390 in 4-directional mode and 370 in 8-directional mode. Since diagonal edges give more choices, the 8-directional MST can have a lower total weight. In both modes, G was reachable from S, and the MST contained 76 vertices and 75 edges.

Overall, the results show that the same maze can produce different optimal solutions depending on the goal: BFS is suitable for shortest unweighted paths, Dijkstra for minimum-cost paths, Edmonds-Karp for maximum flow, and Prim’s algorithm for minimum spanning trees. The

extensions, especially both movement modes and all three cost models, made the comparison clearer.

Known Limitations or Bugs:

The maximum flow algorithm uses Edmonds-Karp. This algorithm is reliable and easier to explain, but its theoretical worst-case complexity is higher than more advanced flow algorithms. It is still acceptable for the required maze size and the project scope, but for very large and dense graphs, it is not the good option at all.

At the time of submission, the main code has no known algorithmic bugs for the required maze. One limitation remains in the optional HTML visualizer: the “Animate Path” button for Subtask D, Maximum Flow is currently not functional. This issue only applies to the visualization extension, and does not affect the correctness of the maximum flow calculation or the output produced by the main program.

Extensions That We Have Applied:

First, the program supports both 4-directional and 8-directional movement.

Second, for the minimum-cost path, we implemented all three cost models: entering cost, leaving cost, and combined cost.

Third, we added command-line options so the user can choose movement mode, run all cost models, generate outputs, export files, and run benchmarks more easily.

Fifth, we implemented benchmarking extension.

Finally, we created an interactive HTML visualizer to display the maze, paths, flow results, MST, and benchmark information more clearly.

AI/Tool Usage Disclosure:

For this assignment, we relied on the free Claude Community edition as an assisting tool. The tool was mainly employed in checking whether the explanation of the algorithm and its complexity are clearly formulated and comprehensible enough.

The implementation of the main program and its testing and debugging, along with results verification, were carried out by the team members. AI was not utilized to generate a solution and was not assistance to us in understanding the algorithms.

In addition, standard development tools including programming language (Python), code editor, version control system (Git), and a web browser to visualize the optional HTML file were used by the team for completing the project.